SG Commute Pulse — Calendar-Aware Singapore Commute Recommender

DLim86 · Singapore Institute of Technology · Data Engineering, Week 2 Assessment

Python · DuckDB · SQL · Parquet · Airflow · Docker · FastAPI · Streamlit · scikit-learn

---

1 What it does

Built a commute recommender that reads my next Google Calendar event and answers one question: when do I need to leave? It replaces checking three separate apps with a single ranked leave-by time, delivered on a live dashboard and through a REST API.

2 The data pipeline

Built a pipeline that combines five sources — Google Calendar, OneMap geocoding and routing, LTA live bus arrivals and train alerts, data.gov.sg weather, and a configured origin location. Raw pulls are saved as dated Parquet files and the cleaned data is loaded into a nine-table DuckDB database, where a SQL view ranks up to three routes using window functions and works out the leave-by time, pushing rainy routes with long walks down the ranking.

3 Making it reliable

Made the pipeline safe to run unattended: failed API calls retry three times with waits of 1, 2 and 4 seconds; re-running ingestion replaces rows instead of duplicating them, proven by unchanged row counts; coordinates are checked against Singapore's bounds; and every run is logged with row counts, duration and errors. Packaged the system as three Docker services sharing one database volume, with all keys and tokens kept out of the image and mounted at runtime.

4 Finding and fixing a modelling mistake

Found that my first model was mostly copying its own input: it predicted total journey time while being given the routing engine's journey time, which carried 87.5% of the model's importance and also capped predictions for long routes. I changed it to predict only the extra delay on top of that estimate, cutting the input to about 5% and removing the ceiling. I also built `bus_headway_gap` — the gap between the next bus and the one after it — as a crowding signal feeding a delay model and a bus-crowding model that score all three routes separately.

5 Measuring whether the model earns its place

Built a daily evaluation that measures the model's average error over a rolling seven days and compares it against using no model at all, with every outcome recording whether it was hand-logged or reconstructed. It currently reports 6.1 minutes against a 10-minute warning threshold, and shows the model is not yet beating the plain routing baseline at 3.7 minutes — a gap I traced to the baseline being scored only on reconstructed outcomes derived from its own schedule.

Repository: github.com/DLim86/SNAIC-sg-commute-pulse · All figures verified against running code. Journey outcomes are hand-logged observations or reconstructed estimates, recorded separately; the system has no GPS trip tracking.